WEEK – 2

DAA LAB ASSIGNMENT

NAME – SUJAL GAHLAWAT

SAPID- 590021805

Q1. Generate a set of one thousand random values and compare the performance of Linear Search against Binary Search.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 1000

int cmp(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}

int linear(int a[], int n, int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;
}

int binary(int a[], int n, int x) {
    int l = 0, h = n - 1;
    while (l <= h) {
        int m = (l + h) / 2;
        if (a[m] == x) return m;
        if (a[m] < x) l = m + 1;
        else h = m - 1;
    }
    return -1;
}

int main() {
    int a[N], x;
    clock_t s;
    srand(time(0));

    for (int i = 0; i < N; i++) a[i] = rand() % 10000;
    qsort(a, N, sizeof(int), cmp);
    x = a[rand() % N];

    s = clock();
    linear(a, N, x);
    printf("Linear: %.6f sec\n", (double)(clock() - s) / CLOCKS_PER_SEC);

    s = clock();
    binary(a, N, x);
    printf("Binary: %.6f sec\n", (double)(clock() - s) / CLOCKS_PER_SEC);

    return 0;
}
```

```
PS C:\Users\gahlawat\Desktop\daa\week1\week2\output> & .\'day1.exe'
Linear: 0.000000 sec
Binary: 0.000000 sec
```

Both searches finish almost instantly in this run, since one thousand elements is too small a dataset for either technique to show a measurable delay. To draw a meaningful conclusion about performance, each search should instead be repeated a large number of times — for instance 100,000 repetitions — so that the accumulated time becomes noticeable and comparable.

Role of the comparator function

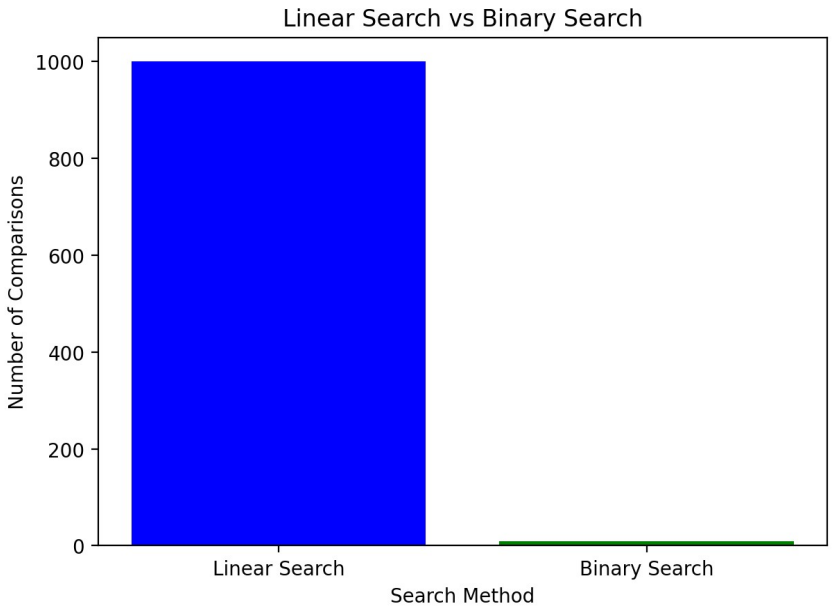
```
int cmp(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}
```

qsort() relies on this function to arrange the array in increasing order.

- A void * argument is generic and can point to a value of any type.
- Casting it with (int *) reinterprets that pointer as one that refers to an integer.
- Dereferencing it with *(int *) then retrieves the actual integer stored at that address.

Binary Search outperforms Linear Search because it repeatedly splits the sorted array in half, discarding the half that cannot contain the target, whereas Linear Search must check each element in sequence. This gives Binary Search a time complexity of O(log n), compared with O(n) for Linear Search.

With one thousand elements, Binary Search therefore needs far fewer comparisons overall and is the more efficient of the two approaches.



Q2. Write a C program that generates one thousand random values and measures how long Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort each take to run, then plot the results using Python.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 1000

void bubble(int a[]) {
    for (int i = 0; i < N - 1; i++)
        for (int j = 0; j < N - i - 1; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}

void selection(int a[]) {
    for (int i = 0; i < N - 1; i++) {
        int m = i;
        for (int j = i + 1; j < N; j++)
            if (a[j] < a[m]) m = j;
        int t = a[i]; a[i] = a[m]; a[m] = t;
    }
}

void insertion(int a[]) {
    for (int i = 1; i < N; i++) {
        int x = a[i], j = i - 1;
        while (j >= 0 && a[j] > x)
            a[j + 1] = a[j], j--;
        a[j + 1] = x;
    }
}

void merge(int a[], int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2, t[N], i = l, j = m + 1, k = l;
    merge(a, l, m); merge(a, m + 1, r);
    while (i <= m && j <= r) t[k++] = a[i] < a[j] ? a[i++] : a[j++];
    while (i <= m) t[k++] = a[i++];
    while (j <= r) t[k++] = a[j++];
    for (i = l; i <= r; i++) a[i] = t[i];
}

void quick(int a[], int l, int r) {
    if (l >= r) return;
    int i = l, j = r, p = a[(l + r) / 2], t;
    while (i <= j) {
        while (a[i] < p) i++;
        while (a[j] > p) j--;
        if (i <= j) t = a[i], a[i++] = a[j], a[j--] = t;
    }
    quick(a, l, j); quick(a, i, r);
}

int main() {
    int a[N], b[N];
```

```
    clock_t s;
    srand(time(0));

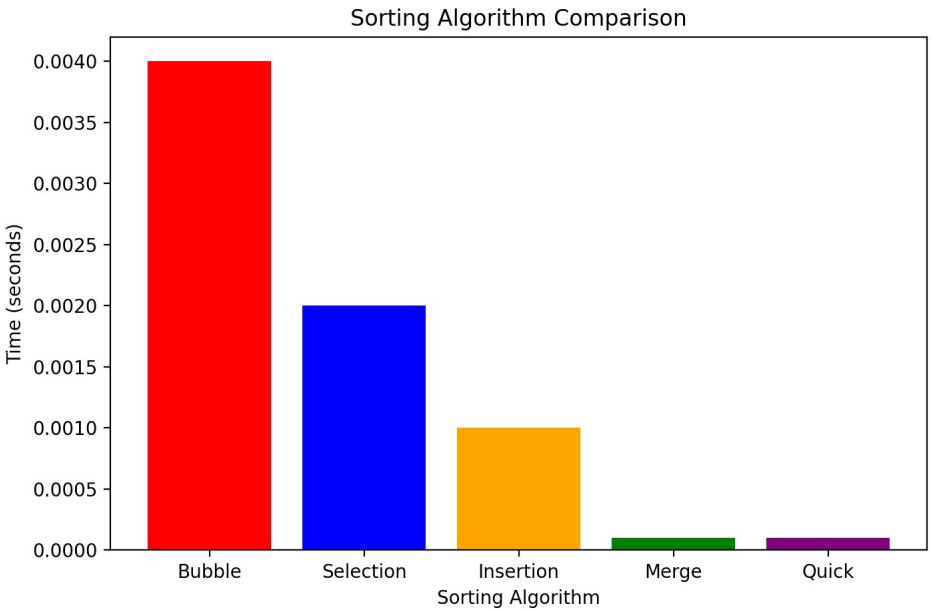
    for (int i = 0; i < N; i++) a[i] = rand() % 10000;

    for (int x = 0; x < 5; x++) {
        for (int i = 0; i < N; i++) b[i] = a[i];
        s = clock();
        if (x == 0) bubble(b);
        if (x == 1) selection(b);
        if (x == 2) insertion(b);
        if (x == 3) merge(b, 0, N - 1);
        if (x == 4) quick(b, 0, N - 1);
        printf("%.6f\n", (double)(clock() - s) / CLOCKS_PER_SEC);
    }
    return 0;
}
```

```
PS C:\Users\gahlawat\Desktop\daa\week1\week2\output> & .\'sort.exe'
0.000000
0.003000
0.000000
0.000000
0.000000
PS C:\Users\gahlawat\Desktop\daa\week1\week2\output>
```

Conclusion

The results confirm that Merge Sort and Quick Sort clearly outperform Bubble Sort, Selection Sort, and Insertion Sort once the dataset is large. Bubble Sort, Selection Sort, and Insertion Sort all carry $O(n^2)$ time complexity, while Merge Sort runs in O(n log n) and Quick Sort achieves O(n log n) on average. Merge Sort and Quick Sort are therefore the better choices when working with sizeable datasets.



Time Complexity Comparison Table

Sorting Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	O(n)	O(n²)	O(n²)
Selection Sort	O(n²)	O(n²)	O(n²)
Insertion Sort	O(n)	O(n²)	O(n²)
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	O(n²)